GRADUATION THESIS (ENGLISH VERSION)

Appendix backup

This document preserves Appendices C–E removed from `main.tex`. Build the main thesis with `main.tex`; compile this file only when the full appendix material is needed.

Contents

APPENDIX A. STUXNET MITM SIMULATION DETAILS

A.1 Stuxnet MITM simulation — supplementary reference

The full Stuxnet MITM walkthrough, including all packet captures, HMI configuration screenshots, Bettercap commands, and parser output, is presented in Section **??** of Chapter 3. This appendix is retained as a cross-reference anchor for the List of Figures and external citations.

APPENDIX B. UPLOAD AND DOWNLOAD TRAFFIC SOURCES

B.1 Upload and Download traffic sources

Chapter **??** describes Upload and Download at the S7comm protocol level. This appendix records why live block-transfer traffic was not captured in the lab and which reference PCAP source was used for rule development instead.

Attempts to simulate program upload and download in the available tooling were unsuccessful because of limitations in the simulation software.

When uploading program code from OpenPLC Editor to OpenPLC Runtime, the program is sent through the OpenPLC Runtime RESTful API rather than using the standard S7comm structure.

Figure **??** shows uploading a program from OpenPLC Editor to OpenPLC Runtime via RESTful API.

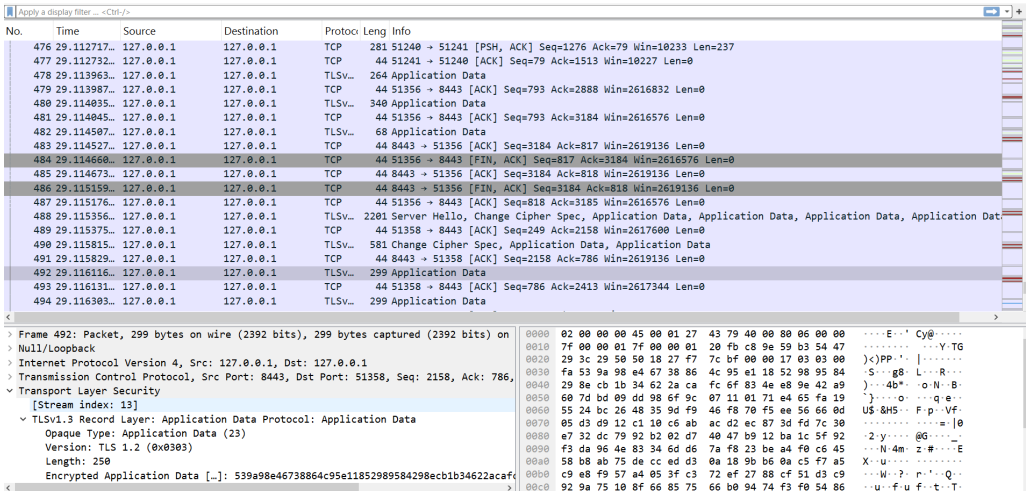

Figure B.1: Uploading a program from OpenPLC Editor to OpenPLC Runtime via RESTful API

OpenPLC Runtime does not support loading programs in the usual Siemens S7comm way.

Another approach is to simulate program transfer on a Siemens PLC through PLCSIM Advanced. In that setup, the program would use the standard S7comm block-transfer sequence.

Step 1 — Set up a virtual switch for Siemens virtual devices:

Figure **??** shows virtual switch setup for Siemens virtual devices.

Step 2 — Run a Siemens PLC in PLCSIM Advanced on the same network as the virtual switch. PLCSIM Advanced only supports simulating Siemens S7-1500

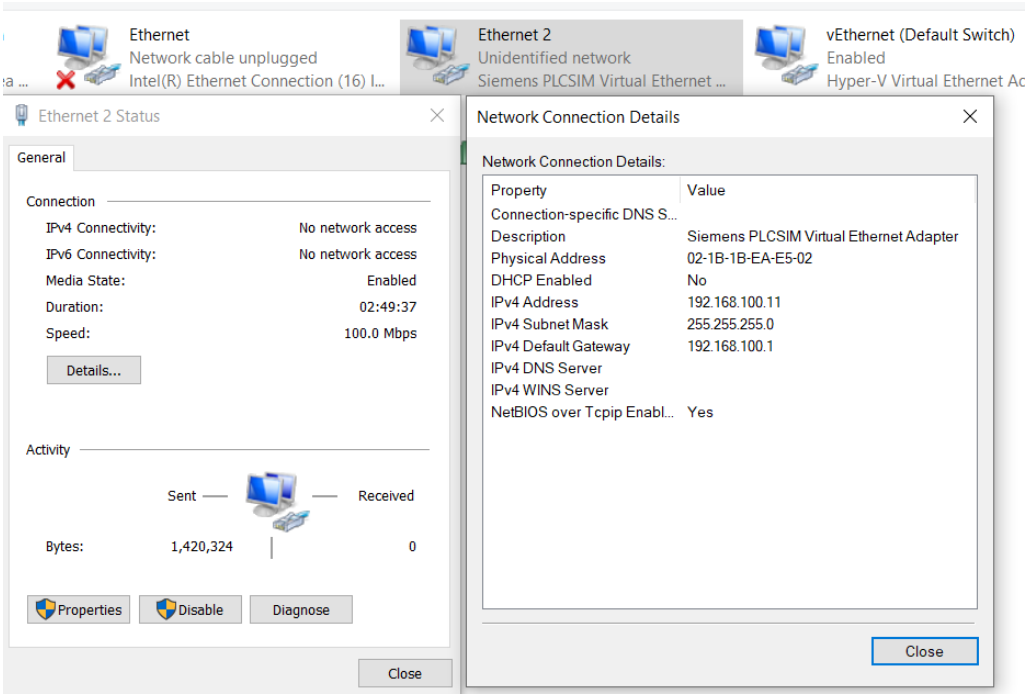

Figure B.2: Virtual switch setup for Siemens virtual devices

PLCs.

Figure **??** shows a Siemens PLC running in PLCSIM Advanced.

Step 3 — Connect the PLC to a WinCC HMI through an IE General connection:

Figure **??** shows the PLC connected to WinCC HMI via an IE General connection card.

Step 4 — Create a PLC program, download it to the PLC, and run a test:

Figure **??** shows the PLC program in RUN mode with PLCSIM Advanced and WinCC HMI.

The screenshot shows the PLC program in RUN mode, the PLC instance in PLCSIM Advanced, and WinCC HMI displaying PLC variable values.

However, when observing packets in Wireshark, Wireshark could not parse S7comm payloads. Investigation showed that S7-1500 uses the S7comm-plus protocol and cannot be forced to use classic S7comm. According to `https://industrialmonitordirect.com/blogs/knowledgebase/s7comm-vs-s7comm-plus-switching-protocol-in-s7-1500-hmi-communication`, enabling `Permit access with PUT/GET communication from remote partners` only allows legacy clients to read selected data; Wireshark still cannot dissect S7comm-plus payloads.

Figure **??** shows Wireshark unable to parse S7comm-plus packets.

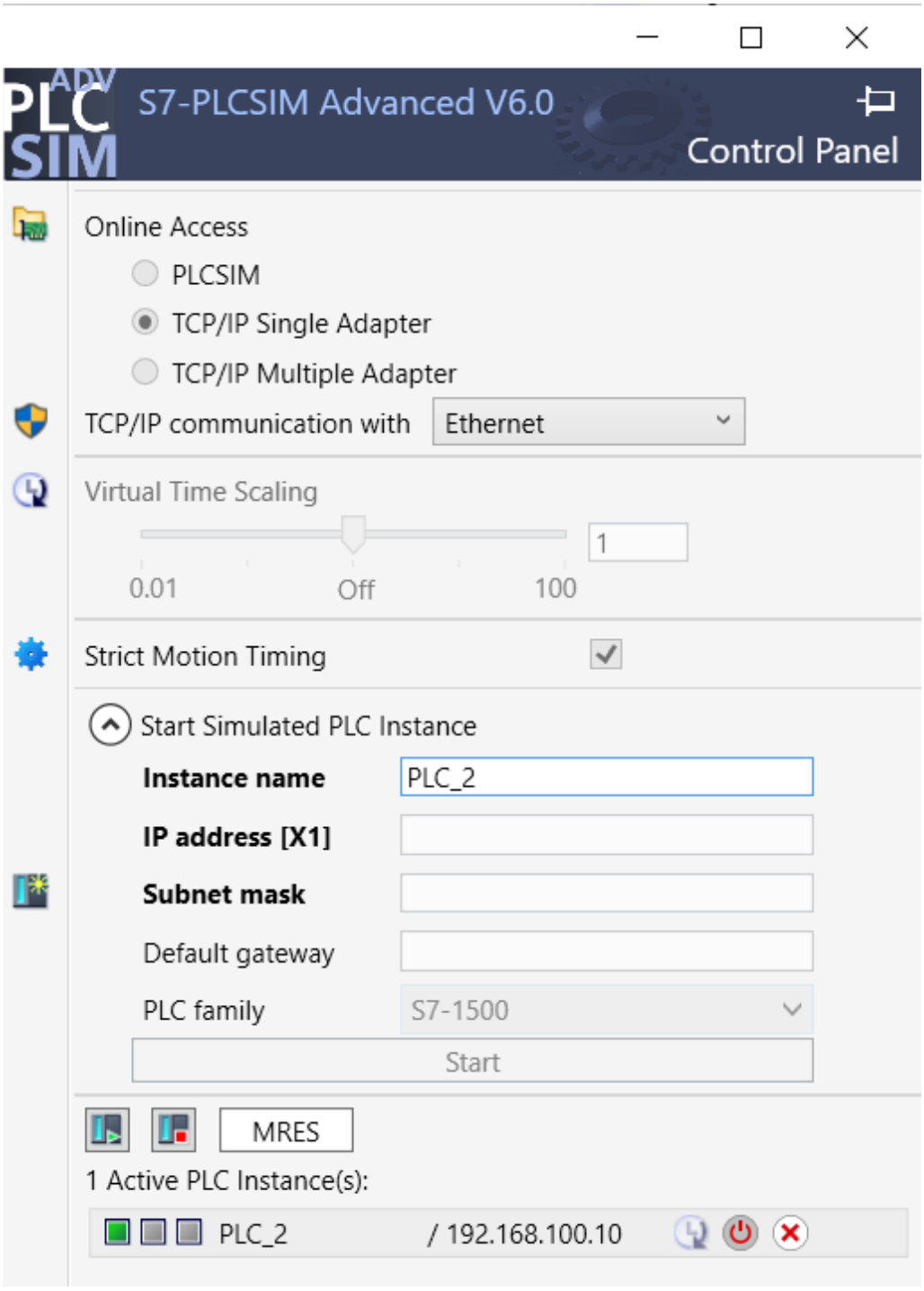

Figure B.3: Siemens PLC running in PLCSIM Advanced

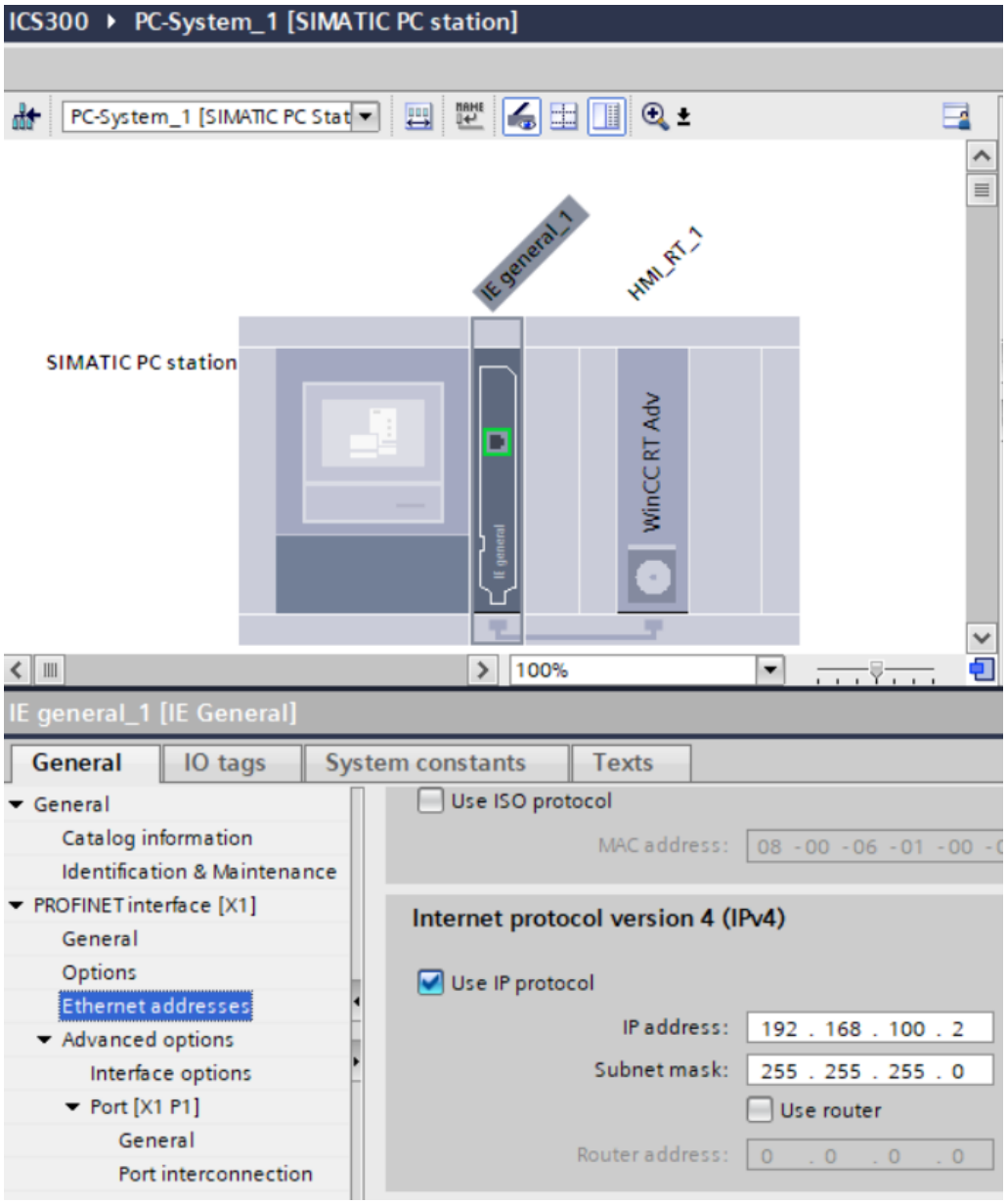

Figure B.4: PLC connected to HMI WinCC via IE general connection card

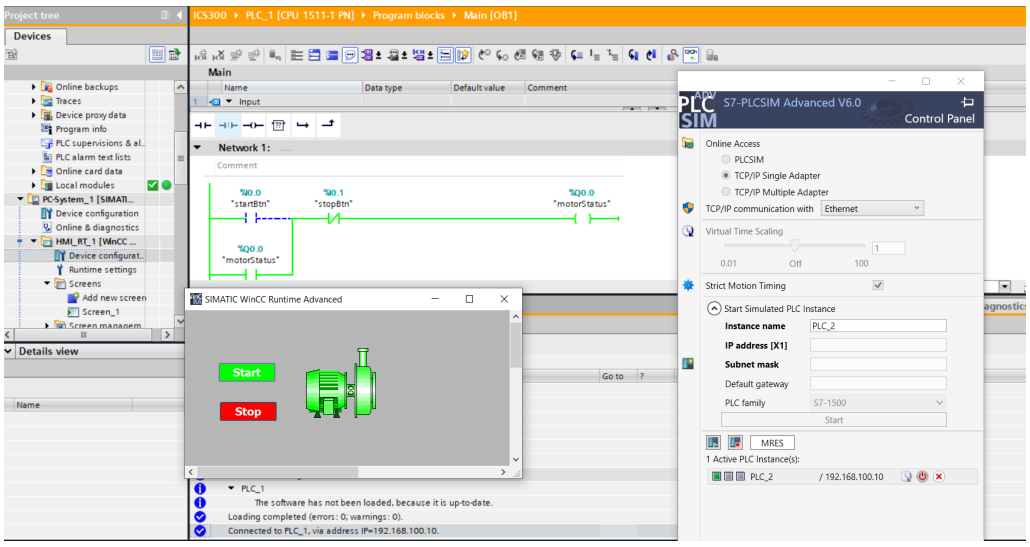

Figure B.5: PLC program in running mode with PLCSIM Advanced and HMI WinCC

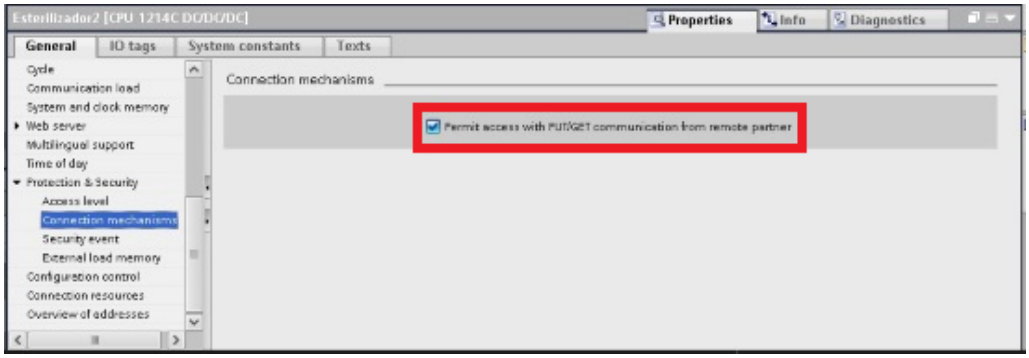

Figure B.6: Wireshark unable to parse S7comm plus packets

Figure **??** shows Wireshark identifying only the outer COTP layer of S7comm-plus packets.

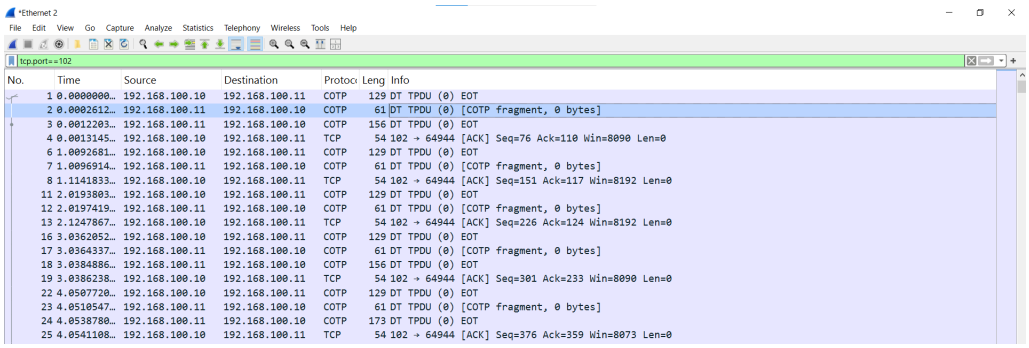

Figure B.7: Wireshark identifying only the outer COTP layer of S7comm plus packets

Wireshark can identify the outer COTP envelope but cannot parse the S7comm-plus PDU carried inside it.

For Upload/Download rule tuning, this project therefore relies on documented sample captures from `https://wiki.wireshark.org/S7comm` rather than on lab-generated block-transfer PCAPs.

APPENDIX C. MALFORMED ATTACK SCRIPT DEPLOYMENT

C.1 Malformed attack script deployment

Chapter **??** describes four groups of malformed S7comm payloads and Table **??** lists the packet modifications used in the lab. This appendix maps each script `--mode` to those modifications and to Suricata SIDs, and records how to run the lab script.

C.1.1 Script modes and Suricata SID mapping

Each sample starts from a valid Setup Communication PDU (same hex as `s7comm_dos.py`) and modifies only one field (or truncates) so results can be compared easily in Wireshark.

Group	`--mode` script	Change vs. valid Setup	SID
1	`tpkt_bad_version`	TPKT byte 0 = `0x02`	1000050
1	`tpkt_len_gt_payload`	TPKT length = `0x0100`, body ≈ 36 B	1000051
1	`tpkt_len_lt_payload`	TPKT length = `0x0008`, full Setup body	1000058
1	`cotp_invalid`	Replace `02 F0 80` with `01 00 00`	1000056
2	`s7_bad_protocol_id`	Byte 7 = `0x31`	1000052
2	`s7_bad_rosctr`	Byte 8 = `0xFF`	1000053
3	`job_bad_opcode`	Byte 17 = `0xFF` (instead of `0xF0`)	1000054
3	`write_var_malformed`	Job `0x05`, 20 B PDU (missing item)	1000057
4	`setup_truncated`	Send only the first 14 bytes	1000055

Table C.1: Malformed script modes and detection rule mapping

C.1.2 Running the lab script

All scenarios are implemented in `attacks/malformed/s7comm_malformed.py`. By default, `--mode all` runs 9 phases sequentially; each phase sends `--repeats` times (default 3) on a new TCP session to port 102, then waits `--pause` seconds (default 6) before the next phase—similar to DoS but without flooding: the goal is to produce clear malformed alerts in `fast.log`.

```
python3 attacks/malformed/s7comm_malformed.py <IP_PLC>
# equivalent to: --mode all --repeats 3 --pause 6
python3 attacks/malformed/s7comm_malformed.py <IP_PLC> --mode s7_bad_rosctr
```

Processing on the wire: attacker → TCP/102 → mirror → Suricata. Unlike `ics_dos.rules`, malformed rules do not use thresholds: each packet matching an anomalous signature may generate one alert (`sid:1000050–1000058`).